

南開大學
Nankai University

计算机学院

计算机组成原理 **lab4** 实验报告

ALU 模块实现

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 5 月 19 日

目录

1 实验目的 **2**

2 实验内容说明 **2**

3 实验原理图 **3**

3.1 ALU 模块原理 3

3.2 4 位操作码设计 3

3.3 改进后顶层模块 4

4 实验步骤 **5**

4.1 `alu.v` 的修改 5

4.2 `alu_display.v` 的修改 6

4.3 `alu.xdc` 的修改 7

4.4 `testbench.v` 的编写与测试 8

5 实验结果分析 **9**

5.1 仿真结果 9

5.2 上板验证 10

5.3 结果分析 14

6 总结感想 **15**

A 源代码 **15**

A.1 `alu.v` 15

A.2 `alu_display.v` 19

A.3 `adder.v` 23

A.4 `alu.xdc` 23

A.5 `testbench.v` 26

1 实验目的

本实验在实验指导手册实验四 ALU 实验的基础上，对原有 ALU 设计进行复现与功能改进。通过本次实验，主要达到以下目的：

1. 熟悉 ALU 的基本结构和工作原理，理解算术运算、逻辑运算、移位运算、比较运算和加载运算在硬件中的实现方式；
2. 掌握使用 Verilog 设计 32 位 ALU 功能模块的方法，并能够通过操作码选择不同的运算结果；
3. 将原实验中 12 位独热码形式的 ALU 操作码改为 4 位二进制操作码，减少控制信号位宽；
4. 修改顶层显示模块 `alu_display.v`，使 ALU 操作码由实验箱上的 4 个拨码开关直接输入，而不再通过 LCD 触摸屏输入；
5. 在保留原有 ALU 运算功能的基础上，新增 SGE、XNOR、LLI 三种运算，分别对应比较运算、位运算和加载运算；
6. 编写仿真文件 `testbench.v`，对改进后的 15 种 ALU 运算进行行为级仿真，并通过波形验证功能正确性；
7. 完成 Vivado 工程的综合、实现和 bit 文件生成，并将设计下载到 FPGA 实验箱上进行上板验证；
8. 熟悉从 Verilog 编码、仿真测试、约束配置到 FPGA 上板验证的完整实验流程。

2 实验内容说明

本次实验以实验指导手册中的 ALU 实验为基础，对原始 ALU 模块和顶层显示模块进行了功能改进。原实验中的 ALU 操作码采用 12 位独热码形式，每一位对应一种 ALU 运算。虽然这种方式译码简单，但控制信号位宽较大，不利于使用实验箱上的少量拨码开关进行直接控制。因此，本实验将原来的 12 位 ALU 操作码修改为 4 位二进制操作码。

改进后的 ALU 操作码范围为 `4'h1` 到 `4'hF`，共支持 15 种运算。其中，前 12 种运算保留原实验中的基本功能，包括加法、减法、有符号比较、无符号比较、按位与、按位或非、按位或、按位异或、逻辑左移、逻辑右移、算术右移和高位加载。在此基础上，本实验新增了三种运算：

- **SGE**：有符号大于等于置位运算。当 `alu_src1` 大于等于 `alu_src2` 时，结果最低位置 1，否则为 0；
- **XNOR**：按位同或运算。对两个源操作数进行按位异或后取反；
- **LLI**：低位加载运算。将 `alu_src2` 的低 16 位加载到结果低 16 位，高 16 位清零。

除了底层 ALU 运算模块的修改外，本实验还对顶层模块 `alu_display.v` 进行了改进。原始设计中，ALU 操作码与两个源操作数均通过 LCD 触摸屏输入；改进后，LCD 触摸屏只负责输入两个源操作数 `SRC_1` 和 `SRC_2`，而 ALU 操作码由实验箱上的 SW0–SW3 四个拨码开关输入。LCD 屏幕上显示 `SRC_1`、`SRC_2`、`ALUOP` 和 `RESUL` 四项内容，便于观察当前输入数据、操作码和运算结果。

在约束文件 `alu.xdc` 中，本实验将 SW0–SW3 绑定到 `sw_alu_control[3:0]`，用于控制 4 位 ALU 操作码；同时将 SW4–SW5 绑定到 `input_sel[1:0]`，用于选择当前 LCD 输入的是源操作数 1 还是源操作数 2。LCD 触摸屏相关引脚约束保持原实验参考设计不变。

完成代码修改后，本实验编写了 `testbench.v` 对 15 种 ALU 运算进行仿真验证。仿真通过后，继续在 Vivado 中完成综合、实现和 bit 文件生成，并将生成的 bit 文件下载到 FPGA 实验箱中。最后，通过 LCD 输入源操作数、拨码开关选择操作码的方式，对 ADD、SGE、XNOR 和 LLI 等典型运算进行了上板验证，证明改进后的 ALU 能够正确工作。

3 实验原理图

3.1 ALU 模块原理

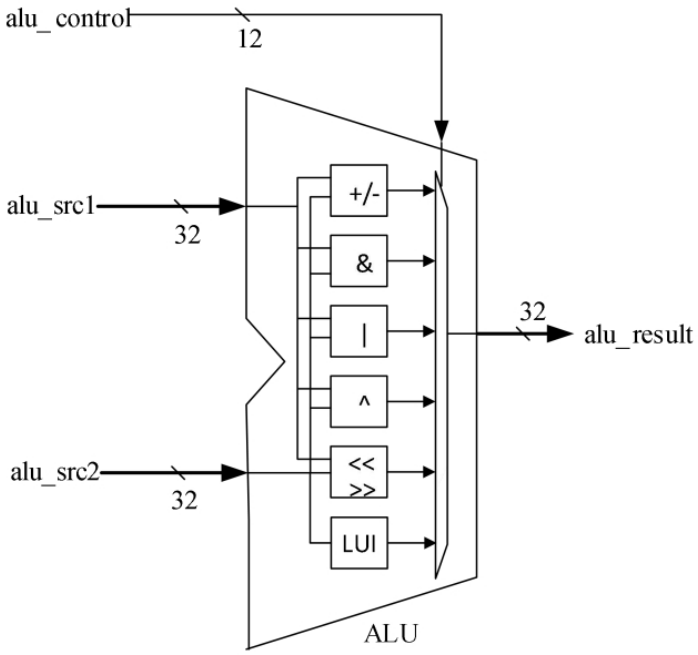


图 3.1: ALU 模块原理图

3.2 4 位操作码设计

原实验中的 ALU 控制信号为 12 位独热码，每一位对应一种 ALU 运算。本实验将其修改为 4 位二进制操作码，通过 `alu_control[3:0]` 表示不同运算。4 位操作码理论上可以表示 16 种状态，本实验使用 `4'h1` 到 `4'hF` 共 15 种操作码，其中 `4'h0` 作为默认空操作状态，输出结果为 0。

改进后的操作码分配如表 1 所示。

表 1: 改进后 ALU 的 4 位操作码设计

操作码	运算名称	功能说明
0001	ADD	加法，src1 + src2
0010	SUB	减法，src1 - src2
0011	SLT	有符号小于置位
0100	SLTU	无符号小于置位
0101	AND	按位与
0110	NOR	按位或非
0111	OR	按位或
1000	XOR	按位异或
1001	SLL	逻辑左移
1010	SRL	逻辑右移
1011	SRA	算术右移
1100	LUI	高 16 位加载
1101	SGE	【新增】有符号大于等于置位
1110	XNOR	【新增】按位同或
1111	LLI	【新增】低 16 位加载

其中，SGE、XNOR 和 LLI 是本次实验新增的三种运算，分别对应题目要求中的比较运算、位运算和加载运算。

3.3 改进后顶层模块

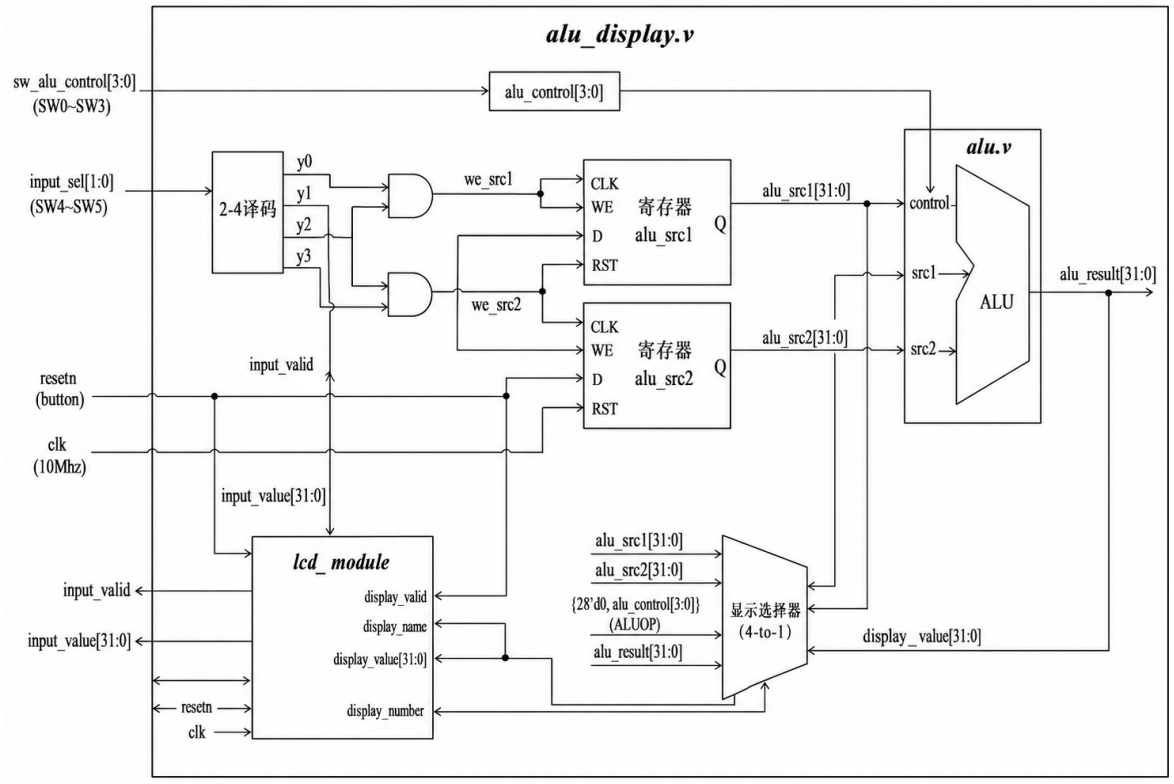


图 3.x 改进后 ALU 顶层模块结构图

图 3.2: 改进后顶层模块原理图

4 实验步骤

本次实验主要完成了底层 ALU 模块、顶层显示模块、约束文件和仿真文件四部分的修改。整体流程为：先修改 `alu.v`，将原来的 12 位独热码控制方式改为 4 位二进制操作码控制；再修改 `alu_display.v`，使操作码由拨码开关输入；随后修改 `alu.xdc` 完成引脚绑定；最后编写 `testbench.v` 对 15 种运算进行仿真，并完成综合、实现、生成 bit 文件和上板验证。

4.1 `alu.v` 的修改

在 `alu.v` 中，首先对模块端口进行了修改。原实验指导手册中的 ALU 控制信号为 12 位独热码，每一位对应一种 ALU 运算。本实验将其修改为 4 位二进制操作码，因此模块端口由原来的

```
input [11:0] alu_control
```

修改为：

```
input [3:0] alu_control
```

这样可以使用 4 个拨码开关直接表示 15 种不同运算，减少了控制信号位宽。

接着，在模块内部根据 `alu_control` 的取值对不同运算进行译码。例如，`4'h1` 表示加法，`4'h2` 表示减法，`4'h3` 表示有符号小于置位，依次类推。对应的译码代码如下：

```
assign alu_add  = (alu_control == 4'h1);
assign alu_sub  = (alu_control == 4'h2);
assign alu_slt  = (alu_control == 4'h3);
assign alu_sltu = (alu_control == 4'h4);
assign alu_and  = (alu_control == 4'h5);
assign alu_nor  = (alu_control == 4'h6);
assign alu_or   = (alu_control == 4'h7);
assign alu_xor  = (alu_control == 4'h8);
assign alu_sll  = (alu_control == 4'h9);
assign alu_srl  = (alu_control == 4'hA);
assign alu_sra  = (alu_control == 4'hB);
assign alu_lui  = (alu_control == 4'hC);
assign alu_sge  = (alu_control == 4'hD);
assign alu_xnor = (alu_control == 4'hE);
assign alu_lli  = (alu_control == 4'hF);
```

原有的 12 种运算在本实验中均被保留，包括加法、减法、有符号小于置位、无符号小于置位、按位与、按位或非、按位或、按位异或、逻辑左移、逻辑右移、算术右移和高位加载。其中，ADD、SUB、SLT、SLTU 仍然复用 `adder` 模块完成加法或减法相关运算。

对于加减法和比较运算，采用如下方式构造加法器输入：

```
assign adder_operand1 = alu_src1;
assign adder_operand2 = alu_add ? alu_src2 : ~alu_src2;
assign adder_cin      = alu_add ? 1'b0 : 1'b1;
```

当执行加法时，加法器完成 `alu_src1 + alu_src2`；当执行减法或比较运算时，加法器完成 `alu_src1`

- alu_src2。这样既保留了原实验中加减运算复用加法器的思想，也便于实现比较运算。

在新增功能方面，本实验增加了 SGE、XNOR 和 LLI 三种运算。SGE 为有符号大于等于置位运算，可以看作 SLT 的取反；XNOR 为按位同或运算；LLI 为低位加载运算。对应代码如下：

```
// SGE: signed(src1) >= signed(src2)
assign sge_result = {31'd0, ~slt_result[0]};

// XNOR: bitwise xnor
assign xnor_result = ~(alu_src1 ^ alu_src2);

// LLI: load low immediate
assign lli_result = {16'b0, alu_src2[15:0]};
```

最后，通过多路选择逻辑根据操作码选择最终输出结果：

```
assign alu_result =
    alu_add  ? add_sub_result :
    alu_sub  ? add_sub_result :
    alu_slt  ? slt_result     :
    alu_sltu ? sltu_result    :
    alu_and  ? and_result     :
    alu_nor  ? nor_result     :
    alu_or   ? or_result      :
    alu_xor  ? xor_result     :
    alu_sll  ? sll_result     :
    alu_srl  ? srl_result     :
    alu_sra  ? sra_result     :
    alu_lui  ? lui_result     :
    alu_sge  ? sge_result     :
    alu_xnor ? xnor_result    :
    alu_lli  ? lli_result     :
               32'd0;
```

通过上述修改，alu.v 完成了从 12 位独热码到 4 位二进制操作码的改进，并实现了共 15 种 ALU 运算。

4.2 alu_display.v 的修改

在顶层显示模块 alu_display.v 中，主要修改目标是将 ALU 操作码输入方式由 LCD 触摸屏输入改为拨码开关输入。原始设计中，alu_control 是一个寄存器，可以通过 LCD 输入；本实验中，操作码直接来自实验箱上的 4 个拨码开关。

首先，在顶层模块端口中新增 4 位拨码开关输入：

```
input [3:0] sw_alu_control
```

然后在模块内部定义 4 位操作码信号，并将其直接连接到拨码开关输入：

```
wire [3:0] alu_control;
assign alu_control = sw_alu_control;
```

这样，alu_control 的值会随着 SW0–SW3 拨码开关的状态实时变化，不再需要通过 LCD 触摸屏输入。

在输入逻辑方面，删除了原来通过 input_sel == 2'b00 输入 alu_control 的 always 块。修改后，LCD 触摸屏只用于输入两个 32 位源操作数。当 input_sel == 2'b10 时，输入数据写入 alu_src1；当 input_sel == 2'b11 时，输入数据写入 alu_src2。对应代码如下：

```
always @(posedge clk)
begin
    if (!resetn)
        alu_src1 <= 32'd0;
    else if (input_valid && input_sel == 2'b10)
        alu_src1 <= input_value;
end

always @(posedge clk)
begin
    if (!resetn)
        alu_src2 <= 32'd0;
    else if (input_valid && input_sel == 2'b11)
        alu_src2 <= input_value;
end
```

在显示逻辑方面，本实验使用 LCD 的前 4 个显示块分别显示源操作数 1、源操作数 2、ALU 操作码和运算结果。显示名称分别为 SRC_1、SRC_2、ALUOP 和 RESUL。其中，ALUOP 显示时将 4 位操作码高位补 0 扩展为 32 位：

```
6'd3:
begin
    display_valid <= 1'b1;
    display_name  <= "ALUOP";
    display_value <= {28'd0, alu_control};
end
```

通过上述修改，顶层模块能够实现通过 LCD 输入两个源操作数，通过拨码开关选择 ALU 运算，并在 LCD 上实时显示操作码和运算结果。

4.3 alu.xdc 的修改

由于顶层模块增加了 sw_alu_control[3:0] 输入端口，并且 input_sel[1:0] 的拨码分配也进行了调整，因此需要修改约束文件 alu.xdc。

本实验中，SW0–SW3 用于控制 4 位 ALU 操作码，对应绑定关系如下：

```
set_property PACKAGE_PIN AC21 [get_ports {sw_alu_control[0]}]
set_property PACKAGE_PIN AD24 [get_ports {sw_alu_control[1]}]
set_property PACKAGE_PIN AC22 [get_ports {sw_alu_control[2]}]
set_property PACKAGE_PIN AC23 [get_ports {sw_alu_control[3]}]
```

其中，sw_alu_control[0] 对应操作码最低位，sw_alu_control[3] 对应操作码最高位。通过这

4 个拨码开关，可以在实验箱上直接输入 0001 到 1111 的 15 种操作码。

同时，为了继续使用 LCD 触摸屏输入两个源操作数，将 SW4 和 SW5 分别绑定到 input_sel[0] 和 input_sel[1]：

```
set_property PACKAGE_PIN AB6 [get_ports {input_sel[0]}]
set_property PACKAGE_PIN W6  [get_ports {input_sel[1]}]
```

当 input_sel = 2'b10 时，LCD 输入值写入 SRC_1；当 input_sel = 2'b11 时，LCD 输入值写入 SRC_2。

除拨码开关相关约束外，LCD 触摸屏相关引脚约束保持原实验参考设计不变，包括 lcd_rst、lcd_cs、lcd_rs、lcd_wr、lcd_rd、lcd_data_io[15:0] 以及触摸屏控制信号等。所有端口的电平标准均设置为 LVCMOS33。

4.4 testbench.v 的编写与测试

为了验证改进后 ALU 的功能正确性，本实验编写了独立的仿真文件 testbench.v。该文件直接实例化 alu.v 模块，而不经过 alu_display.v 和 LCD 外围模块，从而重点验证 ALU 本身的组合逻辑功能。

仿真文件中定义了 3 个输入信号和 1 个输出信号：

```
reg  [3 :0] alu_control;
reg  [31:0] alu_src1;
reg  [31:0] alu_src2;
wire [31:0] alu_result;
```

随后实例化待测模块：

```
alu uut(
    .alu_control(alu_control),
    .alu_src1   (alu_src1),
    .alu_src2   (alu_src2),
    .alu_result (alu_result)
);
```

为了便于重复测试，仿真文件中定义了任务 check_alu。该任务依次输入操作码、两个源操作数和期望结果，等待一段时间后判断实际输出是否与期望值一致：

```
task check_alu;
    input [3 :0] op;
    input [31:0] src1;
    input [31:0] src2;
    input [31:0] expect;
    begin
        alu_control = op;
        alu_src1    = src1;
        alu_src2    = src2;
        #20;

        if (alu_result !== expect)
```

```
            $display("ERROR");
        else
            $display("PASS");
    end
endtask
```

测试过程中，依次对 4'h1 到 4'hF 的 15 种操作码进行验证。例如：

```
check_alu(4'h1, 32'h0000_0005, 32'h0000_0003, 32'h0000_0008);
check_alu(4'hD, 32'h0000_0005, 32'h0000_0003, 32'h0000_0001);
check_alu(4'hE, 32'h0F0F_0F0F, 32'h00FF_00FF, 32'hF00F_F00F);
check_alu(4'hF, 32'h0000_0000, 32'hABCD_1234, 32'h0000_1234);
```

其中，4'hD、4'hE 和 4'hF 分别用于验证本实验新增的 SGE、XNOR 和 LLI 三种运算。仿真时将波形窗口中的信号显示格式设置为十六进制，重点观察 alu_control、alu_src1、alu_src2 和 alu_result。仿真结果表明，15 种操作的输出均与预期结果一致。

5 实验结果分析

5.1 仿真结果

在完成 alu.v 和 testbench.v 的编写后，本实验首先在 Vivado 中进行了行为级仿真。仿真时直接对底层 alu.v 模块进行测试，输入信号包括 alu_control[3:0]、alu_src1[31:0] 和 alu_src2[31:0]，输出信号为 alu_result[31:0]。

仿真过程中，依次将 alu_control 设置为 4'h1 到 4'hF，分别验证 15 种 ALU 运算。仿真波形如图 5.3 所示。

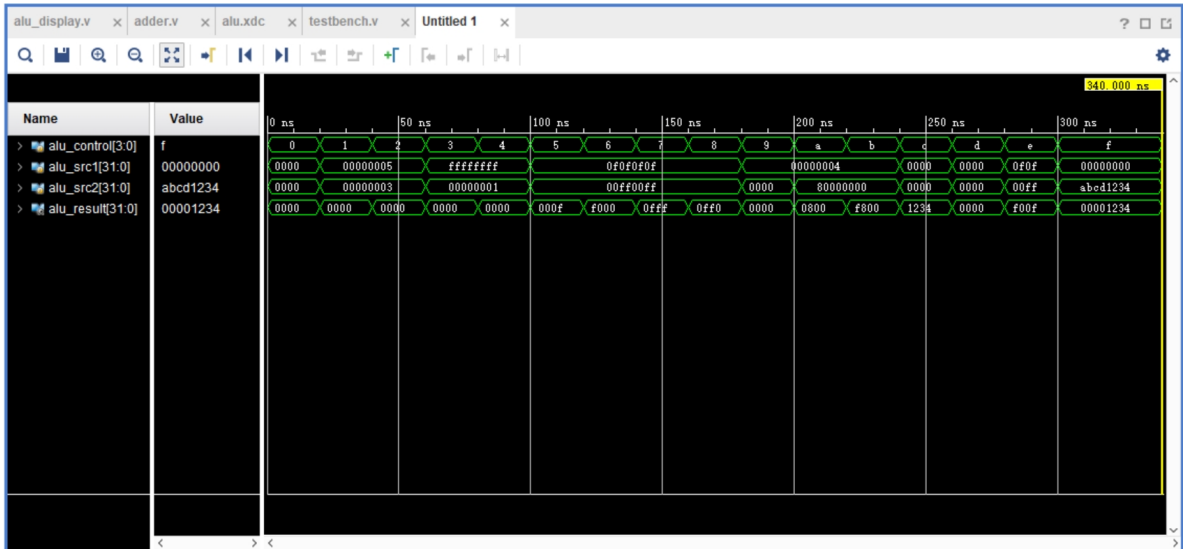


图 5.3: 改进后 ALU 的 15 种操作仿真波形

由图 5.3 可以看出，随着 alu_control 从 0 依次变化到 f，ALU 能够根据不同操作码输出对应运算结果。其中，4'h1 对应加法，4'h2 对应减法，4'h3 到 4'hC 对应原有 ALU 运算，4'hD、4'hE 和 4'hF 分别对应本实验新增的 SGE、XNOR 和 LLI 运算。

仿真中的主要测试结果如下表所示。

表 2: ALU 行为级仿真测试数据

操作码	运算	alu_src1	alu_src2	alu_result
1	ADD	00000005	00000003	00000008
2	SUB	00000005	00000003	00000002
3	SLT	FFFFFFFF	00000001	00000001
4	SLTU	FFFFFFFF	00000001	00000000
5	AND	0F0F0F0F	00FF00FF	000F000F
6	NOR	0F0F0F0F	00FF00FF	F000F000
7	OR	0F0F0F0F	00FF00FF	0FFF0FFF
8	XOR	0F0F0F0F	00FF00FF	0FF00FF0
9	SLL	00000004	00000011	00000110
A	SRL	00000004	80000000	08000000
B	SRA	00000004	80000000	F8000000
C	LUI	00000000	00001234	12340000
D	SGE	00000005	00000003	00000001
E	XNOR	0F0F0F0F	00FF00FF	F00FF00F
F	LLI	00000000	ABCD1234	00001234

综合波形和表中数据可以看出，改进后的 ALU 在 15 种操作码下均能得到正确结果，说明 `alu.v` 的功能修改正确。

5.2 上板验证

本实验选取 ADD、SUB、SLT、SGE、XNOR、SRA 和 LLI 共七种典型运算进行上板验证，其中 ADD、SUB 用于验证基础算术运算，SLT 和 SGE 用于验证比较运算，XNOR 用于验证位运算，SRA 用于验证移位运算，LLI 用于验证加载运算。

实验箱拨码开关功能分配如图 5.4 所示。



图 5.4: 实验箱拨码开关功能分配示意图

为了验证基础算术运算功能，首先测试 ADD 加法运算。设置 SRC_1 = 00000005，SRC_2 = 00000003，并通过拨码开关设置 ALUOP = 00000001。实验箱显示结果如图 5.5 所示。

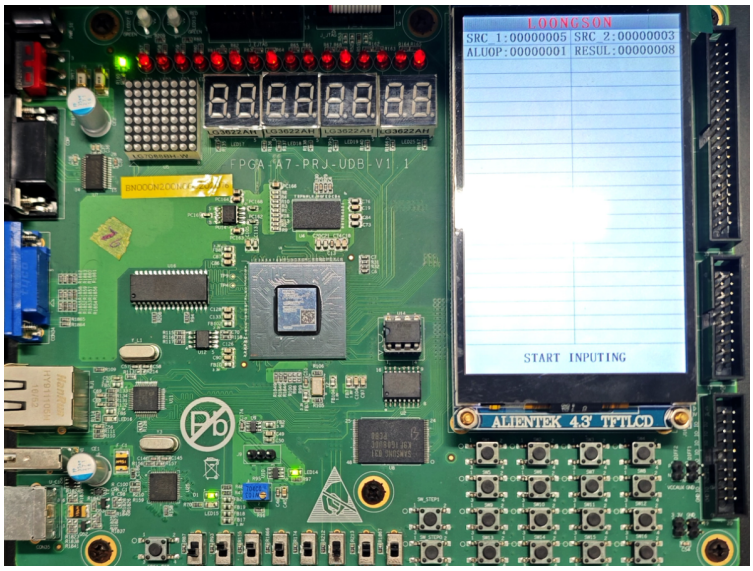


图 5.5: ADD 加法运算上板验证结果

由图 5.5 可以看出，此时 LCD 显示结果为 RESUL = 00000008，与理论计算 $5+3=8$ 一致，说明加法运算上板验证正确。

接着测试 SUB 减法运算。设置 SRC_1 = 00000005，SRC_2 = 00000003，并通过拨码开关设置 ALUOP = 00000002。实验箱显示结果如图 5.6 所示。

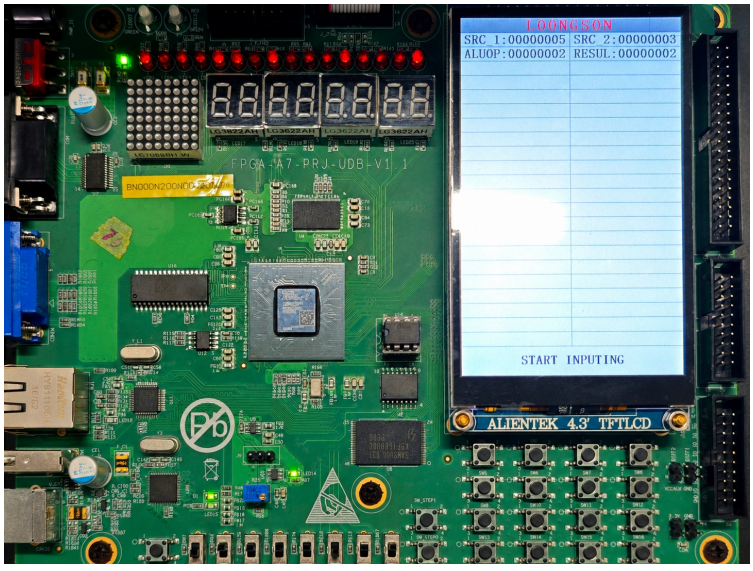


图 5.6: SUB 减法运算上板验证结果

由图 5.6 可以看出，LCD 显示结果为 RESUL = 00000002，与理论计算 $5-3=2$ 一致，说明减法运算功能正确。由于减法在 ALU 内部通过对第二个操作数取反加一并复用加法器实现，因此该结果也验证了加减法复用加法器逻辑的正确性。

接着验证原有的 SLT 有符号小于置位运算。设置 SRC_1 = FFFFFFFF，SRC_2 = 00000001，并通过拨码开关设置 ALUOP = 00000003。其中，FFFFFFFF 按补码解释为 -1，因此有 $-1<1$ 成立，理论

结果应为 00000001。实验箱显示结果如图 5.7 所示。

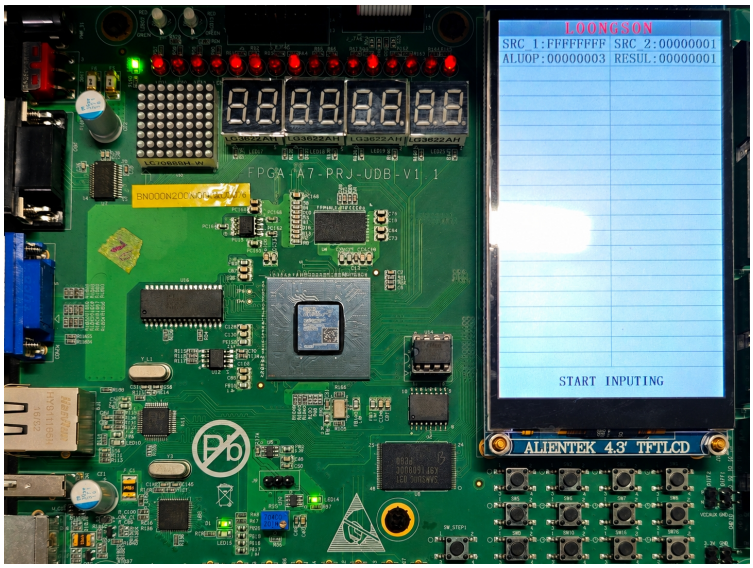


图 5.7: SLT 有符号比较运算上板验证结果

由图 5.7 可以看出，LCD 显示 RESUL = 00000001，与 $-1 < 1$ 的理论判断结果一致，说明有符号小于置位运算正确。

然后验证新增的 SGE 比较运算。设置 SRC_1 = 00000005，SRC_2 = 00000003，并通过拨码开关设置 ALUOP = 0000000D。实验箱显示结果如图 5.8 所示。

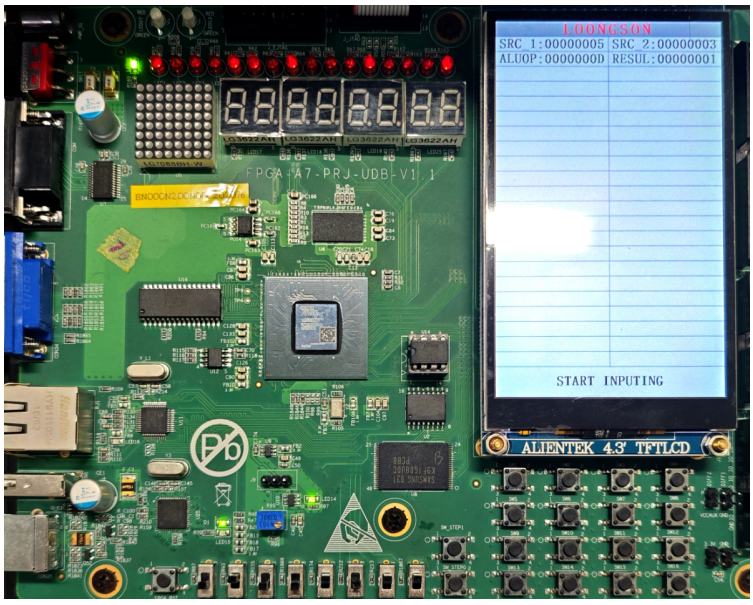


图 5.8: SGE 新增比较运算上板验证结果

由图 5.8 可以看出，LCD 显示 RESUL = 00000001。由于 $5 \geq 3$ 成立，因此 SGE 运算结果应为 1，实验箱显示结果与理论结果一致。

然后验证新增的 XNOR 位运算。设置 SRC_1 = 0F0F0F0F，SRC_2 = 00FF00FF，并设置 ALUOP = 0000000E。实验箱显示结果如图 5.9 所示。

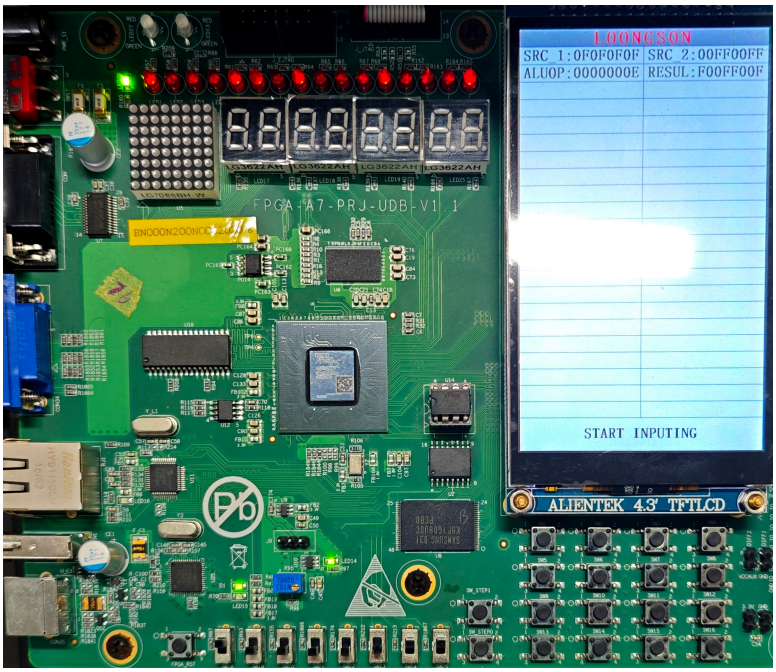


图 5.9: XNOR 新增位运算上板验证结果

由图 5.9 可以看出，LCD 显示 RESUL = F00FF00F。该结果与 0F0F0F0F 和 00FF00FF 按位同或的理论结果一致，说明 XNOR 运算正确。

接着验证 SRA 算术右移运算。设置 SRC_1 = 00000004，SRC_2 = 80000000，并通过拨码开关设置 ALUOP = 0000000B。其中，SRC_1 的低 5 位作为移位量，表示右移 4 位；SRC_2 = 80000000 的最高位为 1，因此算术右移时高位应补 1，理论结果为 F8000000。实验箱显示结果如图 5.10 所示。

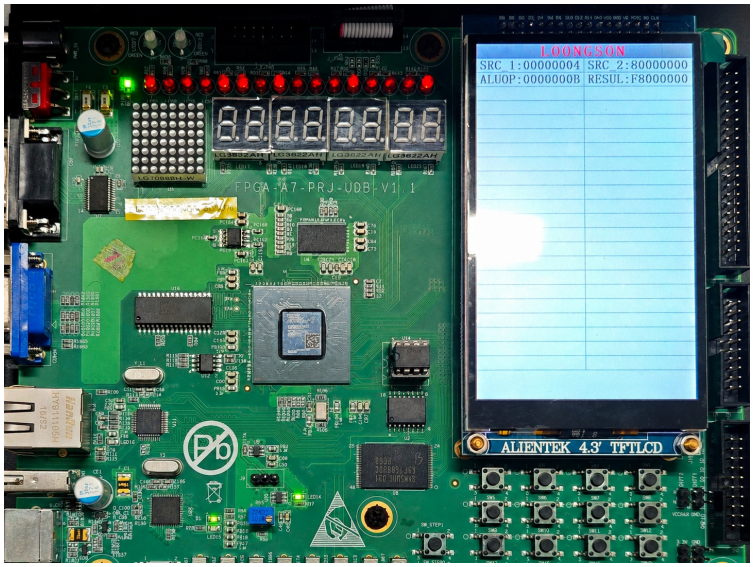


图 5.10: SRA 算术右移运算上板验证结果

由图 5.10 可以看出，LCD 显示 RESUL = F8000000，与算术右移 4 位后高位补符号位 1 的理论结果一致，说明算术右移运算正确。

最后验证新增的 LLI 加载运算。设置 SRC_2 = ABCD1234，并设置 ALUOP = 0000000F。实验箱显示结果如图 5.11 所示。

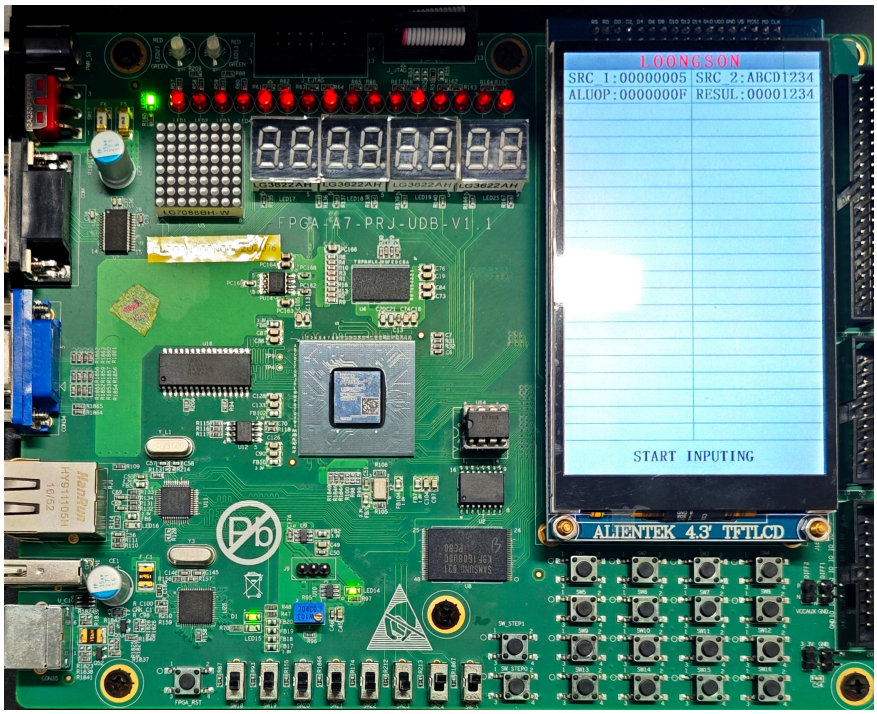


图 5.11: LLI 新增加载运算上板验证结果

由图 5.11 可以看出，LCD 显示 `RESUL = 00001234`。LLI 运算取 `SRC_2` 的低 16 位 `1234`，并将高 16 位清零，因此理论结果为 `00001234`，与上板结果一致。

综上，本实验在上板验证中选取了 ADD、SUB、SLT、SGE、XNOR、SRA 和 LLI 共七种典型运算进行测试。各项测试结果均与理论计算结果一致，说明改进后的 ALU 模块以及顶层显示模块能够在 FPGA 实验箱上正确运行。

5.3 结果分析

综合行为级仿真和实验箱上板验证结果可以看出，改进后的 ALU 模块能够正确完成 15 种操作码对应的运算。行为级仿真中，`alu_control` 从 `4'h1` 到 `4'hF` 依次变化，每一种操作码均得到了预期输出，说明底层 `alu.v` 的组合逻辑设计正确。

从上板验证结果来看，LCD 能够正确显示两个源操作数、当前 ALU 操作码和运算结果，说明 `alu_display.v` 与 LCD 触摸屏模块连接正确。同时，操作码通过 SW0–SW3 拨码开关输入后，LCD 上的 `ALUOP` 能够实时变化，说明 `alu.xdc` 中拨码开关与 `sw_alu_control[3:0]` 的引脚绑定正确。

对于基础 ADD 运算，实验箱结果为 `00000008`，与 $5+3=8$ 一致，说明原有算术运算功能被正确保留。对于新增的 SGE 运算，输入 5 和 3 时输出 `00000001`，说明有符号大于等于置位逻辑正确。对于新增的 XNOR 运算，输出 `F00FF00F`，与按位同或理论结果一致，说明新增位运算功能正确。对于新增的 LLI 运算，输出 `00001234`，说明低 16 位加载逻辑正确。

因此，本实验完成了将原 12 位独热码 ALU 操作码改为 4 位二进制操作码的目标，实现了通过 4 个拨码开关控制 ALU 运算的功能，并在原有运算基础上新增了比较、位运算和加载运算各一种。仿真结果和上板验证结果均表明，本次 ALU 功能改进达到了预期要求。

6 总结感想

通过本次 ALU 模块改进实验，我进一步熟悉了 Verilog 模块化设计、Vivado 工程开发流程以及 FPGA 实验箱上板验证方法。本实验在指导手册原有 ALU 参考设计的基础上进行了改进，将原来的 12 位独热码操作码修改为 4 位二进制操作码，并通过实验箱上的拨码开关直接控制 ALU 运算。通过这一过程，我加深了对 ALU 操作码译码、运算选择和顶层模块接口设计的理解。

在功能实现方面，本实验保留了原有的 12 种 ALU 运算，并新增了 SGE、XNOR 和 LLI 三种运算，分别对应比较运算、位运算和加载运算。新增功能的实现过程中，我体会到合理复用已有逻辑可以简化硬件设计，例如 SGE 可以基于 SLT 的比较结果实现，XNOR 可以由异或结果取反得到。

在顶层模块设计方面，本实验将 LCD 触摸屏输入操作码的方式改为拨码开关输入，使 LCD 只负责输入两个源操作数并显示运算结果。这让我更加清楚地认识到顶层模块不仅要调用底层功能模块，还要负责连接实验箱外设，并组织内部信号用于显示和验证。

实验验证过程中，我先通过 testbench 对 15 种 ALU 运算进行了行为级仿真，然后完成综合、实现和 bit 文件生成，并下载到 FPGA 实验箱上进行上板验证。上板验证中测试了 ADD、SUB、SLT、SGE、XNOR、SRA 和 LLI 等典型运算，结果均与理论值一致，说明改进后的 ALU 能够正确工作。

总体来说，本次实验完整经历了从代码修改、仿真验证到上板测试的过程，使我对 ALU 的设计原理、控制信号编码方式以及 FPGA 实验调试流程有了更深入的理解，也为后续 CPU 数据通路和控制器设计打下了基础。

附录 A 源代码

A.1 alu.v

Listing 1: alu.v 源代码

```
1  `timescale 1ns / 1ps
2  //*************************************************************
3  // > 文件名: alu.v
4  // > 描述  : ALU 模块，改进版
5  // > 修改  : 1. 将原 12 位独热码 alu_control 改为 4 位二进制操作码
6  // >         2. 保留原参考设计中的 12 种 ALU 运算
7  // >         3. 新增 SGE、XNOR、LLI 三种运算
8  // >         4. 加法、减法、SLT、SLTU、SGE 仍复用 adder 模块完成加/减相关运算
9  //*************************************************************
10
11 module alu(
12     input  [3 :0] alu_control,   // 【修改】原来是 [11:0] 独热码，现在改为 4 位二进制操作码
13     input  [31:0] alu_src1,      // ALU 操作数 1
14     input  [31:0] alu_src2,      // ALU 操作数 2
15     output [31:0] alu_result     // ALU 运算结果
16 );
17
18     //============================================================
19     // 1. 4 位操作码译码
20     //============================================================
21     wire alu_add;   // 0001: 加法
```

```
22      wire alu_sub;   // 0010: 减法
23      wire alu_slt;   // 0011: 有符号小于置位
24      wire alu_sltu;  // 0100: 无符号小于置位
25      wire alu_and;   // 0101: 按位与
26      wire alu_nor;   // 0110: 按位或非
27      wire alu_or;    // 0111: 按位或
28      wire alu_xor;   // 1000: 按位异或
29      wire alu_sll;   // 1001: 逻辑左移
30      wire alu_srl;   // 1010: 逻辑右移
31      wire alu_sra;   // 1011: 算术右移
32      wire alu_lui;   // 1100: 高位加载
33
34      // 【新增】三种扩展运算
35      wire alu_sge;   // 1101: 有符号大于等于置位
36      wire alu_xnor;  // 1110: 按位同或
37      wire alu_lli;   // 1111: 低位加载
38
39      assign alu_add  = (alu_control == 4'h1);
40      assign alu_sub  = (alu_control == 4'h2);
41      assign alu_slt  = (alu_control == 4'h3);
42      assign alu_sltu = (alu_control == 4'h4);
43      assign alu_and  = (alu_control == 4'h5);
44      assign alu_nor  = (alu_control == 4'h6);
45      assign alu_or   = (alu_control == 4'h7);
46      assign alu_xor  = (alu_control == 4'h8);
47      assign alu_sll  = (alu_control == 4'h9);
48      assign alu_srl  = (alu_control == 4'hA);
49      assign alu_sra  = (alu_control == 4'hB);
50      assign alu_lui  = (alu_control == 4'hC);
51
52      // 【新增】扩展操作码
53      assign alu_sge  = (alu_control == 4'hD);
54      assign alu_xnor = (alu_control == 4'hE);
55      assign alu_lli  = (alu_control == 4'hF);
56
57
58      //============================================================
59      // 2. 各种运算结果信号
60      //============================================================
61      wire [31:0] add_sub_result;
62      wire [31:0] slt_result;
63      wire [31:0] sltu_result;
64      wire [31:0] and_result;
65      wire [31:0] nor_result;
66      wire [31:0] or_result;
67      wire [31:0] xor_result;
68      wire [31:0] sll_result;
69      wire [31:0] srl_result;
70      wire [31:0] sra_result;
```

```
71      wire [31:0] lui_result;
72
73      // 【新增】扩展运算结果
74      wire [31:0] sge_result;
75      wire [31:0] xnor_result;
76      wire [31:0] lli_result;
77
78
79      //============================================================
80      // 3. 逻辑运算
81      //============================================================
82      assign and_result  = alu_src1 & alu_src2;
83      assign or_result   = alu_src1 | alu_src2;
84      assign nor_result  = ~(alu_src1 | alu_src2);
85      assign xor_result  = alu_src1 ^ alu_src2;
86
87      // 【新增】XNOR: 按位同或
88      assign xnor_result = ~(alu_src1 ^ alu_src2);
89
90
91      //============================================================
92      // 4. 加法器部分
93      //    ADD、SUB、SLT、SLTU、SGE 均复用 adder 模块
94      //============================================================
95      wire [31:0] adder_operand1;
96      wire [31:0] adder_operand2;
97      wire        adder_cin;
98      wire [31:0] adder_result;
99      wire        adder_cout;
100
101     assign adder_operand1 = alu_src1;
102
103     // 【说明】只有 ADD 时直接加 alu_src2;
104     //         其他需要比较或减法的操作, 统一做 alu_src1 - alu_src2
105     assign adder_operand2 = alu_add ? alu_src2 : ~alu_src2;
106     assign adder_cin      = alu_add ? 1'b0 : 1'b1;
107
108     adder adder_module(
109         .operand1(adder_operand1),
110         .operand2(adder_operand2),
111         .cin     (adder_cin),
112         .result  (adder_result),
113         .cout    (adder_cout)
114     );
115
116     assign add_sub_result = adder_result;
117
118
119     //============================================================
```

```
120     // 5. 比较运算
121     //============================================================
122
123     // SLT: 有符号小于置位
124     // 参考原实验手册的符号位判断方法:
125     // src1 为负、src2 为正，则 src1 < src2
126     // src1 和 src2 同号时，看 src1 - src2 的结果符号位
127     assign slt_result[31:1] = 31'd0;
128     assign slt_result[0] =
129         (alu_src1[31] & ~alu_src2[31]) |
130         (~(alu_src1[31] ^ alu_src2[31]) & adder_result[31]);
131
132     // SLTU: 无符号小于置位
133     // 原参考设计中使用 ~adder_cout 判断无符号小于
134     assign sltu_result = {31'd0, ~adder_cout};
135
136     // 【新增】SGE: 有符号大于等于置位
137     // SGE 与 SLT 互补: 若不是 src1 < src2，则 src1 >= src2
138     assign sge_result = {31'd0, ~slt_result[0]};
139
140
141     //============================================================
142     // 6. 移位运算
143     //    移位量使用 alu_src1[4:0]，被移位数为 alu_src2
144     //============================================================
145     assign sll_result = alu_src2 << alu_src1[4:0];
146     assign srl_result = alu_src2 >> alu_src1[4:0];
147     assign sra_result = $signed(alu_src2) >>> alu_src1[4:0];
148
149
150     //============================================================
151     // 7. 加载运算
152     //============================================================
153
154     // LUI: 高位加载，将 src2 低 16 位放到结果高 16 位
155     assign lui_result = {alu_src2[15:0], 16'b0};
156
157     // 【新增】LLI: 低位加载，将 src2 低 16 位放到结果低 16 位，高 16 位清零
158     assign lli_result = {16'b0, alu_src2[15:0]};
159
160
161     //============================================================
162     // 8. 结果选择
163     //============================================================
164     assign alu_result =
165         alu_add  ? add_sub_result :
166         alu_sub  ? add_sub_result :
167         alu_slt  ? slt_result     :
168         alu_sltu ? sltu_result    :
```

```
        alu_and  ? and_result     :
        alu_nor  ? nor_result     :
        alu_or   ? or_result      :
        alu_xor  ? xor_result     :
        alu_sll  ? sll_result     :
        alu_srl  ? srl_result     :
        alu_sra  ? sra_result     :
        alu_lui  ? lui_result     :
        alu_sge  ? sge_result     :   // 【新增】比较运算
        alu_xnor ? xnor_result    :   // 【新增】位运算
        alu_lli  ? lli_result     :   // 【新增】加载运算
                   32'd0;             // 0000 或非法操作码，结果为 0

endmodule
```

A.2 alu_display.v

Listing 2: alu_display.v 源代码

```
//************************************************************************
// > 文件名: alu_display.v
// > 描述   : ALU 显示模块，调用 FPGA 板上的 IO 接口和触摸屏
// > 修改   : 1. 新增 input [3:0] sw_alu_control，用 4 个拨码开关控制 ALU 操作码
// >          2. 删除原来通过 LCD 输入 alu_control 的逻辑
// >          3. LCD 只负责输入 alu_src1 和 alu_src2
// >          4. LCD 显示 SRC_1、SRC_2、ALUOP、RESUL 四项
//************************************************************************

module alu_display(
    // 时钟与复位信号
    input clk,
    input resetn,               // 后缀 "n" 代表低电平有效

    // 拨码开关，用于选择 LCD 输入的是源操作数 1 还是源操作数 2
    // 【修改】现在不再用 input_sel == 00 输入 alu_control
    // 只使用:
    // 2'b10: 输入源操作数 1，即 alu_src1
    // 2'b11: 输入源操作数 2，即 alu_src2
    input [1:0] input_sel,

    // 【新增】4 位 ALU 操作码，来自实验箱上的 4 个拨码开关
    // sw_alu_control[3:0] 对应 alu.v 中的 alu_control[3:0]
    input [3:0] sw_alu_control,

    // 触摸屏相关接口，不需要更改
    output lcd_rst,
    output lcd_cs,
    output lcd_rs,
```

```
30      output lcd_wr,
31      output lcd_rd,
32      inout [15:0] lcd_data_io,
33      output lcd_bl_ctr,
34      inout ct_int,
35      inout ct_sda,
36      output ct_scl,
37      output ct_rstn
38  );
39
40      //============================================================
41      // 1. 调用 ALU 模块
42      //============================================================
43
44      // 【修改】原来这里是 reg [11:0] alu_control;
45      //         现在改为 wire [3:0] alu_control, 并直接接拨码开关
46      wire [3:0] alu_control;
47
48      reg  [31:0] alu_src1;       // ALU 操作数 1
49      reg  [31:0] alu_src2;       // ALU 操作数 2
50      wire [31:0] alu_result;     // ALU 结果
51
52      // 【新增/关键】ALU 操作码来自 4 个拨码开关, 而不是 LCD
53      assign alu_control = sw_alu_control;
54
55      alu alu_module(
56          .alu_control(alu_control),
57          .alu_src1    (alu_src1),
58          .alu_src2    (alu_src2),
59          .alu_result (alu_result)
60      );
61
62
63      //============================================================
64      // 2. 实例化 LCD 触摸屏模块
65      //============================================================
66      reg         display_valid;
67      reg  [39:0] display_name;
68      reg  [31:0] display_value;
69      wire [5 :0] display_number;
70      wire        input_valid;
71      wire [31:0] input_value;
72
73      lcd_module lcd_module(
74          .clk            (clk),          // 10MHz
75          .resetn         (resetn),
76
77          // 调用触摸屏的接口
78          .display_valid  (display_valid),
```

```
79          .display_name   (display_name),
80          .display_value  (display_value),
81          .display_number (display_number),
82          .input_valid    (input_valid),
83          .input_value    (input_value),
84
85          // LCD 触摸屏相关接口，不需要更改
86          .lcd_rst        (lcd_rst),
87          .lcd_cs         (lcd_cs),
88          .lcd_rs         (lcd_rs),
89          .lcd_wr         (lcd_wr),
90          .lcd_rd         (lcd_rd),
91          .lcd_data_io    (lcd_data_io),
92          .lcd_bl_ctr     (lcd_bl_ctr),
93          .ct_int         (ct_int),
94          .ct_sda         (ct_sda),
95          .ct_scl         (ct_scl),
96          .ct_rstn        (ct_rstn)
97      );
98
99
100     //============================================================
101     // 3. 从触摸屏获取输入
102     //============================================================
103
104     // 【修改】删除原来 input_sel == 2'b00 时输入 alu_control 的 always 块。
105     //         现在 LCD 只输入两个源操作数。
106
107     // 当 input_sel 为 2'b10 时，表示输入源操作数 1，即 alu_src1
108     always @(posedge clk)
109     begin
110         if (!resetn)
111         begin
112             alu_src1 <= 32'd0;
113         end
114         else if (input_valid && input_sel == 2'b10)
115         begin
116             alu_src1 <= input_value;
117         end
118     end
119
120     // 当 input_sel 为 2'b11 时，表示输入源操作数 2，即 alu_src2
121     always @(posedge clk)
122     begin
123         if (!resetn)
124         begin
125             alu_src2 <= 32'd0;
126         end
127         else if (input_valid && input_sel == 2'b11)
```

```
128         begin
129             alu_src2 <= input_value;
130         end
131     end
132 
133 
134     //============================================================
135     // 4. 输出到触摸屏显示
136     //============================================================
137     // 触摸屏上共有 44 块显示区域，可显示 44 组 32 位数据。
138     // 这里使用 1~4 号区域：
139     // 1: SRC_1，源操作数 1
140     // 2: SRC_2，源操作数 2
141     // 3: ALUOP，4 位操作码，高 28 位补 0
142     // 4: RESUL，ALU 运算结果
143 
144     always @(posedge clk)
145     begin
146         case (display_number)
147             6'd1:
148             begin
149                 display_valid <= 1'b1;
150                 display_name  <= "SRC_1";
151                 display_value <= alu_src1;
152             end
153 
154             6'd2:
155             begin
156                 display_valid <= 1'b1;
157                 display_name  <= "SRC_2";
158                 display_value <= alu_src2;
159             end
160 
161             6'd3:
162             begin
163                 display_valid <= 1'b1;
164                 // 【修改】原参考代码这里显示 "CONTR"
165                 //         现在改为 "ALUOP"，显示 4 位拨码操作码
166                 display_name  <= "ALUOP";
167                 display_value <= {28'd0, alu_control};
168             end
169 
170             6'd4:
171             begin
172                 display_valid <= 1'b1;
173                 display_name  <= "RESUL";
174                 display_value <= alu_result;
175             end
176 
```

```
177             default:
178             begin
179                 display_valid <= 1'b0;
180                 display_name  <= 40'd0;
181                 display_value <= 32'd0;
182             end
183         endcase
184     end
185 
186 endmodule
```

A.3 adder.v

Listing 3: adder.v 源代码

```
1  `timescale 1ns / 1ps
2  //************************************************************
3  // > 文件名: adder.v
4  // > 描述  : 32 位加法器
5  // > 说明  : ALU 中 ADD、SUB、SLT、SLTU、SGE 会调用该模块
6  //************************************************************
7  
8  module adder(
9      input  [31:0] operand1,
10     input  [31:0] operand2,
11     input         cin,
12     output [31:0] result,
13     output        cout
14 );
15 
16     assign {cout, result} = operand1 + operand2 + cin;
17 
18 endmodule
```

A.4 alu.xdc

Listing 4: alu.xdc 约束文件

```
1  #============================================================
2  # 文件名: alu.xdc
3  # 描述  : ALU 改进实验约束文件
4  # 修改  : 1. 使用 SW0~SW3 作为 4 位 ALU 操作码 sw_alu_control[3:0]
5  #         2. 使用 SW4~SW5 作为 LCD 输入选择 input_sel[1:0]
6  #         3. 保留 LCD 触摸屏相关固定引脚
7  #============================================================
8  
9  
```

```
#------------------------------------------------------------
# 1. 时钟信号
#------------------------------------------------------------
set_property PACKAGE_PIN AC19 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]


#------------------------------------------------------------
# 2. 复位信号，低电平有效
#------------------------------------------------------------
set_property PACKAGE_PIN Y3 [get_ports resetn]
set_property IOSTANDARD LVCMOS33 [get_ports resetn]


#------------------------------------------------------------
# 3. ALU 4 位操作码拨码开关
#    SW0 -> sw_alu_control[0]
#    SW1 -> sw_alu_control[1]
#    SW2 -> sw_alu_control[2]
#    SW3 -> sw_alu_control[3]
#------------------------------------------------------------
set_property PACKAGE_PIN AC21 [get_ports {sw_alu_control[0]}]
set_property PACKAGE_PIN AD24 [get_ports {sw_alu_control[1]}]
set_property PACKAGE_PIN AC22 [get_ports {sw_alu_control[2]}]
set_property PACKAGE_PIN AC23 [get_ports {sw_alu_control[3]}]

set_property IOSTANDARD LVCMOS33 [get_ports {sw_alu_control[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw_alu_control[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw_alu_control[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw_alu_control[3]}]


#------------------------------------------------------------
# 4. LCD 输入选择拨码开关
#    input_sel = 2'b10 时，LCD 输入 alu_src1
#    input_sel = 2'b11 时，LCD 输入 alu_src2
#
#    SW4 -> input_sel[0]
#    SW5 -> input_sel[1]
#------------------------------------------------------------
set_property PACKAGE_PIN AB6 [get_ports {input_sel[0]}]
set_property PACKAGE_PIN W6  [get_ports {input_sel[1]}]

set_property IOSTANDARD LVCMOS33 [get_ports {input_sel[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {input_sel[1]}]


#------------------------------------------------------------
# 5. LCD 触摸屏引脚连接，固定不改
```

```
59  #------------------------------------------------------------
60  set_property PACKAGE_PIN J25 [get_ports lcd_rst]
61  set_property PACKAGE_PIN H18 [get_ports lcd_cs]
62  set_property PACKAGE_PIN K16 [get_ports lcd_rs]
63  set_property PACKAGE_PIN L8  [get_ports lcd_wr]
64  set_property PACKAGE_PIN K8  [get_ports lcd_rd]
65  set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
66
67  set_property PACKAGE_PIN H9  [get_ports {lcd_data_io[0]}]
68  set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
69  set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
70  set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
71  set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
72  set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
73  set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
74  set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
75  set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
76  set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
77  set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
78  set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
79  set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
80  set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
81  set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
82  set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
83
84  set_property PACKAGE_PIN L19 [get_ports ct_int]
85  set_property PACKAGE_PIN J24 [get_ports ct_sda]
86  set_property PACKAGE_PIN H21 [get_ports ct_scl]
87  set_property PACKAGE_PIN G24 [get_ports ct_rstn]
88
89  set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
90  set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
91  set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
92  set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
93  set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
94  set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
95
96  set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
97  set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
98  set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
99  set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
100 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
101 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
102 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
103 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
104 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
105 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
106 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
107 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]

set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
```

A.5 testbench.v

Listing 5: testbench.v 仿真文件

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////
// > 文件名: testbench.v
// > 描述   : ALU 改进实验仿真文件
// > 目的   : 验证 4 位操作码下 15 种 ALU 运算结果是否正确
// > 说明   : 本 testbench 直接测试 alu.v, 不测试 LCD 外围模块
//////////////////////////////////////////////////////////////////

module testbench;

    //================================================================
    // 1. 输入信号
    //================================================================
    reg  [3 :0] alu_control;
    reg  [31:0] alu_src1;
    reg  [31:0] alu_src2;

    //================================================================
    // 2. 输出信号
    //================================================================
    wire [31:0] alu_result;

    //================================================================
    // 3. 实例化待测 ALU
    //================================================================
    alu uut(
        .alu_control(alu_control),
        .alu_src1   (alu_src1),
        .alu_src2   (alu_src2),
        .alu_result (alu_result)
    );

    //================================================================
    // 4. 测试任务: 输入操作码和操作数
```

```
35      //============================================================
36      task check_alu;
37          input [3 :0] op;
38          input [31:0] src1;
39          input [31:0] src2;
40          input [31:0] expect;
41          begin
42              alu_control = op;
43              alu_src1    = src1;
44              alu_src2    = src2;
45              #20;
46
47              if (alu_result !== expect)
48              begin
49                  $display("ERROR: op=%h src1=%h src2=%h result=%h expect=%h",
50                           op, src1, src2, alu_result, expect);
51              end
52              else
53              begin
54                  $display("PASS : op=%h src1=%h src2=%h result=%h",
55                           op, src1, src2, alu_result);
56              end
57          end
58      endtask
59
60      //============================================================
61      // 5. 主测试流程
62      //============================================================
63      initial begin
64          // 初始值
65          alu_control = 4'h0;
66          alu_src1    = 32'h0000_0000;
67          alu_src2    = 32'h0000_0000;
68
69          #20;
70
71          //============================================================
72          // 原有 12 种运算
73          //============================================================
74
75          // 0001 ADD: 5 + 3 = 8
76          check_alu(4'h1, 32'h0000_0005, 32'h0000_0003, 32'h0000_0008);
77
78          // 0010 SUB: 5 - 3 = 2
79          check_alu(4'h2, 32'h0000_0005, 32'h0000_0003, 32'h0000_0002);
80
81          // 0011 SLT: signed(-1) < signed(1), 结果为 1
82          check_alu(4'h3, 32'hFFFF_FFFF, 32'h0000_0001, 32'h0000_0001);
83
```

```
84          // 0100 SLTU: unsigned(FFFFFFFF) < unsigned(1), 结果为 0
85          check_alu(4'h4, 32'hFFFF_FFFF, 32'h0000_0001, 32'h0000_0000);
86
87          // 0101 AND
88          check_alu(4'h5, 32'h0F0F_0F0F, 32'h00FF_00FF, 32'h000F_000F);
89
90          // 0110 NOR
91          check_alu(4'h6, 32'h0F0F_0F0F, 32'h00FF_00FF, 32'hF000_F000);
92
93          // 0111 OR
94          check_alu(4'h7, 32'h0F0F_0F0F, 32'h00FF_00FF, 32'h0FFF_0FFF);
95
96          // 1000 XOR
97          check_alu(4'h8, 32'h0F0F_0F0F, 32'h00FF_00FF, 32'h0FF0_0FF0);
98
99          // 1001 SLL: 0x11 << 4 = 0x110
100         check_alu(4'h9, 32'h0000_0004, 32'h0000_0011, 32'h0000_0110);
101
102         // 1010 SRL: 0x80000000 >> 4 = 0x08000000
103         check_alu(4'hA, 32'h0000_0004, 32'h8000_0000, 32'h0800_0000);
104
105         // 1011 SRA: signed(0x80000000) >>> 4 = 0xF8000000
106         check_alu(4'hB, 32'h0000_0004, 32'h8000_0000, 32'hF800_0000);
107
108         // 1100 LUI: src2[15:0] 放到高 16 位
109         check_alu(4'hC, 32'h0000_0000, 32'h0000_1234, 32'h1234_0000);
110
111         //============================================================
112         // 新增 3 种运算
113         //============================================================
114
115         // 1101 SGE: signed(5) >= signed(3), 结果为 1
116         check_alu(4'hD, 32'h0000_0005, 32'h0000_0003, 32'h0000_0001);
117
118         // 1110 XNOR: ~(0F0F0F0F ^ 00FF00FF) = F00FF00F
119         check_alu(4'hE, 32'h0F0F_0F0F, 32'h00FF_00FF, 32'hF00F_F00F);
120
121         // 1111 LLI: src2[15:0] 放到低 16 位, 高 16 位清零
122         check_alu(4'hF, 32'h0000_0000, 32'hABCD_1234, 32'h0000_1234);
123
124         #20;
125         $display("ALU simulation finished.");
126         $stop;
127     end
128
129 endmodule
```